

Lab 5: Task-Priority kinematic control (2A)



HANDS-ON INTERVENTION: Vehicle-Manipulator Systems

Report by: PRAVIN OLI (u1999699), IFROS
(E-mail: pravin.oli.08@gmail.com)

Code Group : Pravin Oli and Gebreslassie Gebrechekos Gebrezgabhier

Abstract—This lab focused on controlling a 3-link planar robot arm using a Task-Priority control approach. We extended the control algorithm to handle both regular tasks (like reaching a target) and safety tasks (like avoiding obstacles or staying within joint limits). The lab was divided into two exercises. In the first, the robot avoided obstacles while trying to reach a goal position. In the second, we made sure the robot's joint stayed within a safe range. The results showed that the robot could follow its goal while also respecting safety rules, thanks to the task-priority structure.



1 INTRODUCTION

IN this lab, we worked on improving the Recursive Task-Priority (TP) kinematic control algorithm by adding support for inequality tasks. These tasks are important for keeping the robot safe—for example, by avoiding obstacles or staying within joint limits. While the basic TP algorithm focuses on goals like moving the robot's end-effector to a specific point (equality tasks), real robots often need to stay within safe zones as they move.

The lab had two main parts:

Exercise 1, was about obstacle avoidance. We controlled a simulated 3-link planar robot arm and made sure it reached a target position without hitting circular obstacles in its workspace.

Exercise 2, focused on joint limits. We added a constraint to keep the first joint of the robot within a safe range during the motion. The **Task-Priority algorithm** enables the manipulator to execute multiple tasks simultaneously, prioritizing them in a hierarchical manner.

1.1 Objectives

The objective of this lab are:

- To extend the Recursive Task-Priority (TP) control algorithm to handle set-based (inequality) tasks, such as obstacle avoidance and joint limit enforcement, in addition to standard equality-based tasks.
- To implement and evaluate inequality tasks in a simulated 3-link planar manipulator, using real-time visualizations and performance plots to demonstrate safe and goal-directed motion under task constraints.

- Erasmus Mundus Joint Masters In Intelligent Field Robotic Systems, University of Girona, Spain.

2 PLANAR ROBOTIC MANIPULATOR

A planar manipulator operates within a two-dimensional plane, typically the XY-plane, where each joint motion and link transformation is constrained to that plane. These manipulators often employ revolute (rotational) or prismatic (translational) joints to achieve desired configurations.

2.1 3-Link Planar Manipulator with Revolute Joints

The manipulator studied in this work consists of three revolute joints (3R), forming a serial kinematic chain. All joints rotate about the Z-axis, and each link lies entirely within the XY-plane.

The configuration (or state) of the manipulator is defined by the vector of joint angles:

$$\mathbf{q} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

where:

- θ_1 is the angle between the first link and the base X-axis,
- θ_2 is the relative angle between the first and second links,
- θ_3 is the relative angle between the second and third links.

The forward kinematics are computed using the Denavit-Hartenberg (DH) convention, which gives the relationship between adjacent links. Each transformation from link $i - 1$ to i is given by:

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

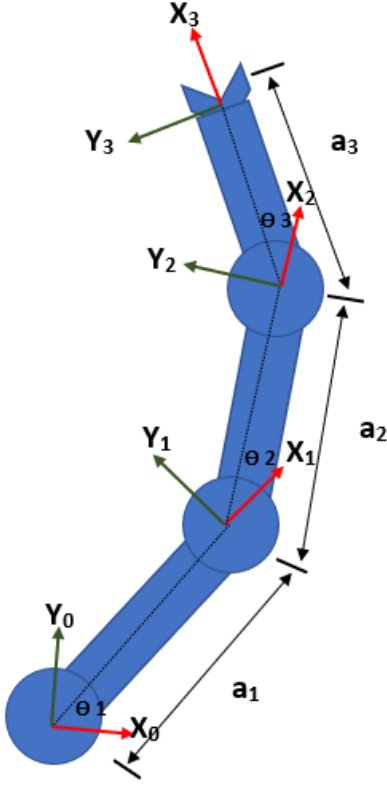


Fig. 1: 3-Link Planar Manipulator Structure

In our implementation, the 3-link manipulator uses zero twist angles ($\alpha_i = 0$) and zero offsets ($d_i = 0$), simplifying the transformation matrices and focusing all complexity in the joint rotations and link lengths. The actual link parameters used in the simulation are:

i	a_i (m)	α_i (rad)	d_i (m)	θ_i (rad)
1	0.6	0	0	$q_1 = 0.2$
2	0.5	0	0	$q_2 = 0.2$
3	0.4	0	0	$q_3 = 0.2$

TABLE 1: Denavit-Hartenberg Parameters for 3R Planar Manipulator

2.2 Inequality Tasks: Concept

Inequality tasks, also known as set-based tasks, are designed to keep a task variable within a safe range or outside of dangerous zones. These tasks are not constantly active but are triggered only when the system is about to reach unsafe regions. This approach allows the robot to prioritize its main objective (like moving the end-effector to a desired position) while also maintaining safety.

The general task variable is given by:

$$\sigma_i = \sigma_i(\mathbf{q}) \in \mathbb{R}^{m_i}$$

where $\mathbf{q}$ is the joint configuration vector. The error is defined as:

$$\tilde{\sigma}_i = \sigma_{i,d} - \sigma_i$$

and the task velocity is:

$$\dot{\mathbf{x}}_i = \dot{\sigma}_i + K_i \tilde{\sigma}_i$$

Activation and deactivation thresholds are used to switch the task on and off smoothly, which helps to avoid sudden control changes (known as chatter).

2.3 Inequality Tasks: Joint Limits Task

The joint limits task ensures that a joint's position stays within defined minimum and maximum values. This is crucial to prevent the robot from damaging itself or exceeding its mechanical boundaries.

The task variable is:

$$\sigma_{li} = q_i$$

and it must remain in:

$$\mathcal{S}_{li} = [q_{i,\min}, q_{i,\max}]$$

The Jacobian for this task is simply a row vector with a 1 at the position of the joint being limited:

$$J_{li} = [0, \dots, 1, \dots, 0]$$

Task activation is controlled using thresholds:

$$a_{li}(\mathbf{q}) = \begin{cases} -1, & \text{if } q_i \geq q_{i,\max} - \alpha_{li} \\ 1, & \text{if } q_i \leq q_{i,\min} + \alpha_{li} \\ 0, & \text{otherwise (within safe range)} \end{cases}$$

When activated, the task generates a constant velocity away from the limit.

2.4 Inequality Tasks: Obstacle Avoidance Task

The obstacle avoidance task is designed to keep the robot's end-effector away from circular obstacles in the workspace. The distance between the end-effector and the obstacle center is monitored. If the robot gets too close, the task activates and pushes the robot away.

The task variable is defined as the distance:

$$\sigma_r(\mathbf{q}) = \|\eta_{ee}(\mathbf{q}) - P\|$$

The task becomes active when:

$$\|\eta_{ee} - P\| \leq r_\alpha$$

and deactivates when:

$$\|\eta_{ee} - P\| \geq r_\delta$$

The task error is a normalized vector pointing away from the obstacle:

$$\tilde{\sigma}_r = \frac{\eta_{ee} - P}{\|\eta_{ee} - P\|}$$

The Jacobian for this task is the translational part of the Jacobian of the link (typically the end-effector).

2.5 Extension of the Recursive Task-Priority Algorithm

The recursive Task-Priority algorithm is used to handle multiple tasks with different priorities. For set-based tasks, we extend the algorithm to check whether a task is active or not before applying it.

Initially, the joint velocity vector ζ is zero. Then, for each task:

- If it is active, the algorithm projects it into the null space of the previous tasks. - If not active, the joint velocity remains unchanged.

The updated recursive formula becomes:

$$\zeta_i = \zeta_{i-1} + \bar{J}_i^\dagger (a_i \dot{x}_i - J_i \zeta_{i-1})$$

$$P_i = P_{i-1} - \bar{J}_i^\dagger \bar{J}_i$$

where $\bar{J}_i = J_i P_{i-1}$ and a_i is the activation flag.

This ensures that safety tasks (like avoiding obstacles or limiting joints) are handled first, and lower-priority tasks only operate in the remaining available motion space.

Algorithm 1 Recursive Task Priority Control with Inequality Tasks

Require: List of tasks $\{J_i(q), \dot{x}_i(q), a_i(q)\}$, where $i \in \{1, \dots, k\}$

Ensure: Quasi-velocities $\zeta_k \in \mathbb{R}^n$

```

1: Initialize:  $\zeta_0 \leftarrow 0^n$ ,  $P_0 \leftarrow I^{n \times n}$ 
2: for  $i = 1$  to  $k$  do
3:   if  $a_i(q) \neq 0$  then
4:     Compute projected Jacobian:  $\bar{J}_i(q) \leftarrow J_i(q) P_{i-1}$ 
5:     Compute task velocity:
6:        $\zeta_i \leftarrow \zeta_{i-1} + \bar{J}_i^\dagger(q) (a_i(q) \dot{x}_i(q) - J_i(q) \zeta_{i-1})$ 
7:   Update null-space projector:
8:      $P_i \leftarrow P_{i-1} - \bar{J}_i^\dagger(q) \bar{J}_i(q)$ 
9:   else
10:     $\zeta_i \leftarrow \zeta_{i-1}$ 
11:     $P_i \leftarrow P_{i-1}$ 
12:   end if
13: end for
14: return  $\zeta_k$ 

```

3 IMPLEMENTATION

This lab is implemented in three Python scripts: **lab2 robotics.py** (the same as the previous lab), **lab5 robotics.py** and **Exercise1.py** and **Exercise2.py**

3.1 lab2 robotics.py

This is the same as in the previous lab. I have added pseudo-inverse calculation.

```

def PseudoInverseJacob(J):
    return np.linalg.pinv(J)

```

3.2 lab5 robotics.py

Below is an implementation of lab5 robotics.py:

```

1 from lab2_robotics import * # Includes numpy import
2
3 # Wraps angles into [0, 2pi)
4 def wrap_angle(angle):
5     return angle % (2.0 * np.pi)
6
7 def jacobianLink(T, revolute, link): # Needed in
8     Exercise 2
9     '''
10    Function builds a Jacobian for the end-
11    effector of a robot,
12    described by a list of kinematic
13    transformations and a list of joint
14    types.
15
16    Arguments:
17    T (list of Numpy array): list of
18    transformations along the kinematic
19    chain of the robot (from the base frame)
20    revolute (list of Bool): list of flags
21    specifying if the corresponding joint is
22    a revolute joint
23    link(integer): index of the link for which
24    the Jacobian is computed
25
26    Returns:
27    (Numpy array): end-effector Jacobian
28    '''
29    # Code almost identical to the one from
30    lab2_robotics...
31
32    J = np.zeros((6, len(T)-1)) # Empty Jacobian (6
33    x N) where N - number of joints
34    O = T[-1][0:3, 3] # End-effector position
35
36    for i in range(link):
37        z = T[i][0:3, 2] # z-axis of joint i
38        o = T[i][0:3, 3] # position of joint
39        i
40        if revolute[i]:
41            J[:, i] = np.cross(z, (O - o)) #
42            Linear velocity
43            J[3:, i] = z #
44            Angular velocity
45        else:
46            J[:, i] = z
47            J[3:, i] = np.zeros(3)
48    return J
49
50 '''
51 Class representing a robotic manipulator.
52 '''
53 class Manipulator:
54     '''
55     Constructor.
56
57     Arguments:
58     d (Numpy array): list of displacements along
59     Z-axis
60     theta (Numpy array): list of rotations
61     around Z-axis
62     a (Numpy array): list of displacements along
63     X-axis
64     alpha (Numpy array): list of rotations
65     around X-axis
66     revolute (list of Bool): list of flags
67     specifying if the corresponding joint is
68     a revolute joint
69
70     '''
71     def __init__(self, d, theta, a, alpha, revolute)
72         :
73         self.d = d
74         self.theta = theta
75         self.a = a

```

```

54     self.alpha = alpha
55     self.revolute = revolute
56     self.dof = len(revolute) # Number of
57                               degrees of freedom
58     self.q = np.zeros(self.dof).reshape(-1, 1)
59     # Joint variables initialized to zero
60     self.update(0.0, 0.0) # Initialize
61                               transformations
62
63 def update(self, dq, dt):
64     """
65     Method that updates the state of the robot.
66
67     Arguments:
68     dq (Numpy array): a column vector of joint
69     velocities
70     dt (double): sampling time
71
72     self.q += dq * dt # Integrates joint
73     velocities over time step
74     for i in range(len(self.revolute)):
75         if self.revolute[i]:
76             self.theta[i] = self.q[i] # Updates
77             revolute joints
78         else:
79             self.d[i] = self.q[i] # Updates
80             prismatic joints
81     self.T = kinematics(self.d, self.theta, self
82                          .a, self.alpha) # Recompute forward
83                          kinematics
84
85 def drawing(self):
86     """
87     Method that returns the characteristic
88     points of the robot.
89
90     return robotPoints2D(self.T)
91
92 def getEEJacobian(self):
93     """
94     Method that returns the end-effector
95     Jacobian.
96
97     return jacobian(self.T, self.revolute)[:2,
98     :2]
99
100 def getOrientationJacobian(self, link):
101     """
102     Method that returns the orientation jacobian
103     Jacobian.
104
105     return self.getLinkJacobian(link)[5, :] #
106     Orientation Jacobian
107
108 def getConfigurationJacobian(self, link):
109     """
110     Method that returns the configuration
111     Jacobian.
112
113     J = np.block([[self.getLinkJacobian(link)
114     [0:2, :].reshape(2,3)],
115     [self.getOrientationJacobian(
116     link).reshape(1,3)]]])
117
118     return J
119
120 def getEETransform(self):
121     """
122     Method that returns the end-effector
123     transformation.
124
125     return self.T[-1]
126
127 def getJointPos(self, joint):
128     """
129     Method that returns the position of a
130     selected joint.
131
132     Argument:
133     joint (integer): index of the joint
134
135     Returns:
136     (double): position of the joint
137
138     return self.q[joint]
139
140 def getLinkTransform(self, link):
141     """
142     Method that returns the transformation of a
143     selected link.
144
145     Argument:
146     link (integer): index of the link
147
148     Returns:
149     (Numpy array): transformation of the link
150
151     return self.T[link]
152
153 def getLinkJacobian(self, link):
154     """
155     Method that returns the Jacobian of a
156     selected link.
157
158     Argument:
159     link (integer): index of the link
160
161     Returns:
162     (Numpy array): Jacobian of the link
163
164     return jacobianLink(self.T, self.revolute,
165     link)
166
167 def getDOF(self):
168     """
169     Method that returns number of DOF of the
170     manipulator.
171
172     return self.dof
173
174 """
175 Base class representing an abstract Task.
176 """
177 class Task:
178     """
179     Constructor.
180
181     Arguments:
182     name (string): title of the task
183     desired (Numpy array): desired sigma (goal)
184
185     def __init__(self, name, desired, link, v_ff,
186     K):
187         self.name = name # task title
188         self.sigma_d = desired # desired sigma
189         self.v_ff = v_ff # feedforward velocity
190         self.K = K # gain matrix
191         self.link = link
192         self.activate_flag = 1 # flag indicating if
193         the task is active

```

```

183     '''
184     Method updating the task variables (abstract
185     ).
186     Arguments:
187     robot (object of class Manipulator):
188         reference to the manipulator
189     '''
190     def isActive(self):
191         return True
192
193     def getName(self):
194         return self.name
195
196     def update(self, robot):
197         pass
198
199
200     def setDesired(self, value):
201         '''
202         Method setting the desired sigma.
203
204         Arguments:
205         value(Numpy array): value of the desired
206             sigma (goal)
207         self.sigma_d = value
208
209
210     def getDesired(self):
211         '''
212         Method returning the desired sigma.
213         '''
214         return self.sigma_d
215
216
217     def getJacobian(self):
218         '''
219         Method returning the task Jacobian.
220         '''
221         return self.J
222
223
224     def getError(self):
225         '''
226         Method returning the task error (tilde sigma)
227         ).
228         '''
229         return self.err
230
231     def setGain(self, K):
232         '''
233         Method setting the gain matrix K.
234
235         Arguments:
236         K(Numpy array): gain matrix
237         '''
238         self.K = K
239
240     def getGain(self):
241         '''
242         Method returning the gain matrix K.
243         '''
244         return self.K
245
246     def SetFeedforwardVelocity(self, value):
247         '''
248         Method setting the feedforward velocity.
249
250         Arguments:
251         value(Numpy array): value of the feedforward
252             velocity
253         self.v_ff = value
254
255     def getFeedforwardVelocity(self):
256
257
258     Method returning the feedforward velocity.
259     '''
260     return self.v_ff
261
262 class Position2D(Task):
263     '''
264     Subclass of Task, representing the 2D position
265     task.
266     '''
267     def __init__(self, name, desired, link=3, v_ff = np.zeros((2,1)), K = np.eye(2)):
268         super().__init__(name, desired, link, v_ff, K)
269         self.J = np.zeros((2,3)) # Initialize
270             Jacobian
271         self.err = np.zeros((2,1)) # Initialize
272             error vector
273
274     def update(self, robot):
275         # self.J = robot.getEEJacobian()
276         # self.err = self.sigma_d - robot.get ()
277         [0:2,3].reshape(2,1)
278
279         self.J = robot.getLinkJacobian(self.link)
280         [0:2, :] # Update task Jacobian
281         desired = self.getDesired().reshape(2,1)
282         expected = robot.getLinkTransform(self.link)
283             [0:2,-1].reshape(2,1)
284         self.err = desired - expected # Update task
285             error
286
287 class Orientation2D(Task):
288     '''
289     Subclass of Task, representing the 2D
290     orientation task.
291     '''
292     def __init__(self, name, desired, link=3, v_ff = np.zeros((1,1)), K = np.eye(1)):
293         super().__init__(name, desired, link, v_ff, K)
294         self.J = np.zeros((1,3))
295         self.err = np.zeros((1,1)) # Initialize with
296             proper dimensions
297
298     def update(self, robot):
299
300         self.J = robot.getOrientationJacobian(self.
301             link).reshape(1,3) # Update task
302             Jacobian
303         T1 = robot.getLinkTransform(self.link)[1,0]
304         T0 = robot.getLinkTransform(self.link)[0,0]
305         sigma = np.arctan2(T1,T0).reshape(1,1)
306         self.err = np.array(self.sigma_d - sigma ).
307             reshape(1,1)# Update task error
308
309 class Configuration2D(Task):
310     '''
311     Subclass of Task, representing the 2D
312     configuration task.
313     '''
314     def __init__(self, name, desired, link=3, v_ff = np.zeros((3,1)), K = np.eye(3)):
315         super().__init__(name, desired, link, v_ff, K)
316         self.J = np.zeros((3,3))
317         self.err = np.zeros((1,1)) # Initialize with
318             proper dimensions
319
320     def update(self, robot):
321         self.J = robot.getConfigurationJacobian(self.
322             link).reshape(3,3)# Update task
323             Jacobian
324         T1 = robot.getLinkTransform(self.link)[1,0]
325         T0 = robot.getLinkTransform(self.link)[0,0]
326         sigma_or= np.arctan2(T1,T0).reshape(1,1)

```

```

310     sigma_pos =robot.getLinkTransform(self.link)
311         [0:2,-1].reshape(2,1)
312     sigma = np.block([[sigma_pos], [sigma_or]])
313         reshape(3,1)
314     self.err = self.sigma_d - sigma # Update
315         task error
316 class JointPosition(Task):
317     '''
318     Subclass of Task, representing the joint
319         position task.
320     def __init__(self, name, desired, link=1, v_ff=
321         None, K=None):
322         """
323         Constructor for JointPosition Task.
324
325         Arguments:
326         name (string): Name of the task
327         desired (numpy array): Desired joint
328             position
329         link (integer): Index of the joint (default
330             is 1)
331         v_ff (numpy array): Feedforward velocity
332             vector (default is 0)
333         K (numpy array): Gain matrix (default is
334             identity)
335         """
336         # Ensure correct dimensions for feedforward
337         velocity and gain matrix
338         if v_ff is None:
339             v_ff = np.zeros((1,1)) # Default
340                 feedforward velocity (1x1)
341         if K is None:
342             K = np.eye(1) # Default gain matrix (1
343                 x1)
344         super().__init__(name, desired, link, v_ff,
345             K) # Call parent constructor
346         self.link = link # Joint index
347         self.J = np.zeros((1, 3)) # Initialize
348             Jacobian (1x3)
349         self.err = np.zeros((1, 1)) # Initialize
350             error (1x1)
351     def update(self, robot):
352         """
353         Update method to compute Jacobian and error
354             for joint position control.
355
356         Arguments:
357         robot (Manipulator): The manipulator object
358         """
359         # Set Jacobian based on which joint is being
360             controlled
361         self.J = np.zeros((1, 3)) # Reset Jacobian
362         if self.link < 3: # Ensure within valid
363             joint range
364             self.J[0, self.link - 1] = 1 # Assign
365                 to the correct joint
366
367         # Compute error between desired and current
368             joint position
369         self.err = self.sigma_d - robot.getJointPos(
370             self.link - 1)
371
372 class Obstacle2D(Task):
373     def __init__(self, name, obstacle_pose ,
374         obstacle_th , link =3):
375         #super().__init__(name , obstacle_pose )
376         super().__init__(name, obstacle_pose, link,
377             np.zeros((2,1)), np.eye(2))
378         self.J =np.zeros((2,3))# Initialize with
379             proper dimensions
380
381     self.err =np.zeros((2,1)) # Initialize with
382         proper dimensions
383     self.obstacle_pose = obstacle_pose
384     self.obstacle_th = obstacle_th
385     self.activation_th = obstacle_th[0]
386     self.deactivation_th = obstacle_th[1]
387     self.active_flag = 0
388     self.link = link
389     self.v_ff = np.zeros((2,1))
390     self.K = np.eye(2)
391
392     def update(self, robot):
393         self.J = robot.getLinkJacobian(self.link)
394         [0:2, :] # jacobian
395     end_effector_pose = robot.getEETransform()
396         [0:2,3].reshape(2,1) # end effector pose
397     self.distance = end_effector_pose - self.
398         obstacle_pose # distance between end
399         effector and obstacle pose
400     dir_vector_len = np.linalg.norm(self.
401         distance) # length of the distance
402         vector
403     self.err = self.distance / dir_vector_len #
404         update task error
405
406     if(self.active_flag == 0 and dir_vector_len
407         < self.obstacle_th[0]):
408         self.active_flag = 1 # activate the task
409
410     elif(self.active_flag == 1 and
411         dir_vector_len > self.obstacle_th[1]):
412         self.active_flag = 0 # deactivate the
413             task
414
415     def isTaskActive(self):
416         # check if the task is active
417         if(self.active_flag == 0):
418             return False
419         else:
420             return True
421
422     def distance_to_obstacle(self):
423         # get distance to obstacle
424         self.dis_to_obs = np.linalg.norm(self.
425             distance ) - self.activation_th
426         return self.dis_to_obs
427
428 class JointLimit(Task):
429     def __init__(self, name, velo_limit,
430         velo_threshold, link=1):
431         v_ff = np.zeros((1,1))
432         K = np.eye(1)
433         desired = np.zeros((1,1)) # Placeholder;
434             not used for inequality
435         super().__init__(name, desired, link, v_ff,
436             K)
437
438         self.J = np.zeros((1, 3))
439         self.err = np.zeros((1, 1))
440         self.q_min = velo_limit[0]
441         self.q_max = velo_limit[1]
442         self.activation_th = velo_threshold[0]
443         self.deactivation_th = velo_threshold[1]
444         self.active_flag = 0
445         self.link = link
446
447     def update(self, robot):
448         ''' update the task state '''
449         self.J = np.array([[1,0,0]]).reshape((1,3))
450             # jacobian
451         self.err = np.array([5]).reshape(1,1) #
452             Initialize with proper dimensions
453
454         qi = robot.getJointPos(self.link-1) # joint
455             position

```

```

422         if(self.active_flag == 0 and qi >= self.
423             q_max - self.activation_th):
424             self.active_flag = -1
425         elif(self.active_flag == 0 and qi <= self.
426             q_min + self.activation_th):
427             self.active_flag = 1
428         elif(self.active_flag == -1 and qi <= self.
429             q_max - self.deactivation_th):
430             self.active_flag = 0
431         elif(self.active_flag == 1 and qi >= self.
432             q_min + self.deactivation_th):
433             self.active_flag = 0
434         self.err = self.err*self.active_flag #
435             update task error
436     def isTaskActive(self):
437         ''' check if the task is active '''
438         if(self.active_flag == 0):
439             return False
440         else:
441             return True
442
443     Position2D("End-effector position", np.
444         array([1.0, 0.5]).reshape(2,1)
445     ]
446     '''
447     # Task hierarchy definition
448     tasks = obstacle_tasks + [
449         # JointLimit("Joint limit", velocity_limit,
450             velo_thershold = [0.05 , 0.15] ),
451         Position2D("End-effector position", np.array
452             ([1.0, 0.5]).reshape(2,1))
453     ]
454     num_obstacles = len(obstacle_tasks)
455     err_counter = 0 # Error counter
456     error_vector = np.zeros((0 ,len(tasks))) # Error
457         vector
458     obstacle_dist = np.zeros((0,num_obstacles))
459
460     # Simulation params
461     dt = 1.0/60.0
462
463     # Drawing preparation
464     fig = plt.figure()
465     ax = fig.add_subplot(111, autoscale_on=False, xlim
466         =(-2, 2), ylim=(-2,2))
467     ax.set_title('Simulation')
468     ax.set_aspect('equal')
469     ax.grid()
470     ax.set_xlabel('x[m]')
471     ax.set_ylabel('y[m]')
472
473     ax.add_patch(patch.Circle(obstacle_pos.flatten(),
474         obstacle_r, color='blue', alpha=0.3))
475     ax.add_patch(patch.Circle(obstacle_pos2.flatten(),
476         obstacle_r2, color='orange', alpha=0.3))
477     ax.add_patch(patch.Circle(obstacle_pos3.flatten(),
478         obstacle_r3, color='green', alpha=0.3))
479
480     line, = ax.plot([], [], 'o-', lw=2)
481     path, = ax.plot([], [], 'c-', lw=1)
482     point, = ax.plot([], [], 'rx')
483     PPx, PPy = [], []
484
485     # Simulation initialization
486     def init():
487         global sigma_endeff
488         line.set_data([], [])
489         path.set_data([], [])
490         point.set_data([], [])
491         sigma_endeff = np.random.uniform(-1.2, 1.2, (2,
492             1))
493
494     # Desired end-effector configuration
495     sigma_config = np.block([[sigma_endeff], [np.pi
496         /2]]).reshape(3,1)
497
498     # endeffector task and configuration task
499     for task in tasks:
500         if(task.name == "End-effector position"):
501             task.setDesired(sigma_endeff)
502         elif(task.name == "End-effector
503             configuration"):
504             task.setDesired(sigma_config)
505
506     return line, path, point
507
508     # Simulation loop
509     def simulate(t):
510         global tasks, robot, PPx, PPy, err_counter,
511             obstacle_dist, sigma_endeff, error_vector
512
513         ### Recursive Task-Priority algorithm (w/set-
514             based tasks)
515         # The algorithm works in the same way as in Lab4
516         # The only difference is that it checks if a

```

3.3 Exercise1.py

Below is an implementation of Exercise1.py:

```

1 from lab5_robotics import * # Includes numpy import
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as anim
4 import matplotlib.patches as patch
5
6 # Robot model
7 d = np.zeros(3) #
8     displacement along Z-axis
9 theta = np.array([0.2, 0.2, 0.2]).reshape(3,1) #
10     rotation around Z-axis
11 alpha = np.zeros(3) # rotation
12     around X-axis
13 a = np.array([0.6, 0.5, 0.4]) #
14     displacement along X-axis
15 revolute = [True, True, True] # flags
16     specifying the type of joints
17 robot = Manipulator(d, theta, a, alpha, revolute) #
18     Manipulator object
19
20 # Task hierarchy definition
21 obstacle_pos = np.array([0.0, 1.0]).reshape(2,1)
22 obstacle_r = 0.5
23
24 obstacle_pos2 = np.array([-0.5, -0.75]).reshape(2,1)
25 obstacle_r2 = 0.25
26
27 obstacle_pos3 = np.array([0.75, -0.5]).reshape(2,1)
28 obstacle_r3 = 0.3
29
30 # obstacle task definition
31 obstacle_tasks = [
32     Obstacle2D("Obstacle avoidance", obstacle_pos,
33         np.array([obstacle_r , obstacle_r+0.05])),
34     Obstacle2D("Obstacle avoidance", obstacle_pos2,
35         np.array([obstacle_r2 , obstacle_r2+0.05])),
36     Obstacle2D("Obstacle avoidance", obstacle_pos3,
37         np.array([obstacle_r3 , obstacle_r3+0.05])),
38 ]
39
40 # Desired end-effector position
41 sigma_endeff = np.zeros((2,1))
42
43 '''
44 tasks = [
45     Obstacle2D("Obstacle avoidance",
46         obstacle_pos, np.array([obstacle_r,
47         obstacle_r+0.05])),
48
49     Position2D("End-effector position", np.
50         array([1.0, 0.5]).reshape(2,1)
51     ]
52     '''
53     # Task hierarchy definition
54     tasks = obstacle_tasks + [
55         # JointLimit("Joint limit", velocity_limit,
56             velo_thershold = [0.05 , 0.15] ),
57         Position2D("End-effector position", np.array
58             ([1.0, 0.5]).reshape(2,1))
59     ]
60     num_obstacles = len(obstacle_tasks)
61     err_counter = 0 # Error counter
62     error_vector = np.zeros((0 ,len(tasks))) # Error
63         vector
64     obstacle_dist = np.zeros((0,num_obstacles))
65
66     # Simulation params
67     dt = 1.0/60.0
68
69     # Drawing preparation
70     fig = plt.figure()
71     ax = fig.add_subplot(111, autoscale_on=False, xlim
72         =(-2, 2), ylim=(-2,2))
73     ax.set_title('Simulation')
74     ax.set_aspect('equal')
75     ax.grid()
76     ax.set_xlabel('x[m]')
77     ax.set_ylabel('y[m]')
78
79     ax.add_patch(patch.Circle(obstacle_pos.flatten(),
80         obstacle_r, color='blue', alpha=0.3))
81     ax.add_patch(patch.Circle(obstacle_pos2.flatten(),
82         obstacle_r2, color='orange', alpha=0.3))
83     ax.add_patch(patch.Circle(obstacle_pos3.flatten(),
84         obstacle_r3, color='green', alpha=0.3))
85
86     line, = ax.plot([], [], 'o-', lw=2)
87     path, = ax.plot([], [], 'c-', lw=1)
88     point, = ax.plot([], [], 'rx')
89     PPx, PPy = [], []
90
91     # Simulation initialization
92     def init():
93         global sigma_endeff
94         line.set_data([], [])
95         path.set_data([], [])
96         point.set_data([], [])
97         sigma_endeff = np.random.uniform(-1.2, 1.2, (2,
98             1))
99
100     # Desired end-effector configuration
101     sigma_config = np.block([[sigma_endeff], [np.pi
102         /2]]).reshape(3,1)
103
104     # endeffector task and configuration task
105     for task in tasks:
106         if(task.name == "End-effector position"):
107             task.setDesired(sigma_endeff)
108         elif(task.name == "End-effector
109             configuration"):
110             task.setDesired(sigma_config)
111
112     return line, path, point
113
114     # Simulation loop
115     def simulate(t):
116         global tasks, robot, PPx, PPy, err_counter,
117             obstacle_dist, sigma_endeff, error_vector
118
119         ### Recursive Task-Priority algorithm (w/set-
120             based tasks)
121         # The algorithm works in the same way as in Lab4
122         # The only difference is that it checks if a

```

```

    task is active.
100   ###
101   dof = robot.getDof() # Get robot degrees of
    freedom
102   P = np.eye(dof) ## Initialize null-space
    projector
103   err_acc = np.zeros((1,0))
104   obs_dis = np.zeros((1 , 0))
105   dq = np.zeros((dof,1)) # Initialize joint
    velocity vector

106
107   for index, task in enumerate(tasks): # Loop over
    tasks
108       task.update(robot) # Update task state
109       J = task.getJacobian() # Compute augmented
    Jacobian
110       # Compute task velocity
111       if(task.isTaskActive()):# check activation
    of the task
112           Jbar= J @ P # Compute augmented Jacobian
113           dq = dq + DLS(Jbar,0.1)@( task.
    getFeedforwardVelocity() +
114               task.getGain()@task.getError
    () - J@dq)
115           P = P - np.linalg.pinv(Jbar)@Jbar #
    Update null-space projector

116
117       new_error = np.linalg.norm(task.getError())
118       if task.name == "Obstacle avoidance":
119           obs_dis = np.block([obs_dis, task.
    distance_to_obstacle()])
120       err_acc = np.block([err_acc, new_error])

121
122   # Update robot
123   robot.update(dq, dt)
124   error_vector = np.block([[error_vector], [
    err_acc]])
125   obstacle_dist = np.block([[obstacle_dist], [
    obs_dis]])
126   err_counter += 1

127
128   # Update drawing
129   PP = robot.drawing()
130   line.set_data(PP[0,:], PP[1,:])
131   PPx.append(PP[0,-1])
132   PPy.append(PP[1,-1])
133   path.set_data(PPx, PPy)
134   point.set_data(sigma_endeff[0], sigma_endeff[1])
135
136   return line, path, point

137
138 # Run animation
139 animation = anim.FuncAnimation(fig, simulate, np.
    arange(0, 10, dt),
140                               interval=10, blit=
    True, init_func=
    init, repeat=
    True)

141 plt.show()

142
143 # Plot errors and distances after simulation
144 total_time = np.arange(0, err_counter * dt, dt)
145 fig = plt.figure()
146 ax2 = fig.add_subplot(111, autoscale_on=False, xlim
    =(0, err_counter * dt),
147                       ylim=(error_vector.min() -
    0.1, error_vector.max() +
    0.1))
148 ax2.set_title(f'Task-Priority Control: {len(tasks)}
    Tasks, K={tasks[0].getGain()[0][0]}')
149 ax2.set_xlabel('Time (s)')
150 ax2.set_ylabel('Value')
151 ax2.grid()
152
153 for i in range(num_obstacles):
154     ax2.plot(total_time, obstacle_dist[:, i], label=
    f"d_{i+1} Distance to Obstacle")

```

```

155
156 for i, task in enumerate(tasks):
157     if task.name != "Obstacle avoidance":
158         ax2.plot(total_time, error_vector[:, i],
    label=f"{task.name} Error")
159
160 ax2.legend()
161 plt.show()

```

3.4 Exercise2.py

Below is an implementation of Exercise2.py:

```

1 from lab5_robotics import * # Includes numpy import
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as anim
4 import matplotlib.patches as patch
5
6 # Robot model
7 d = np.zeros(3) #
    displacement along Z-axis
8 theta = np.array([0.2, 0.2, 0.2]).reshape(3,1) #
    rotation around Z-axis
9 alpha = np.zeros(3) # rotation
    around X-axis
10 a = np.array([0.6, 0.5, 0.4]) #
    displacement along X-axis
11 revolute = [True, True, True] # flags
    specifying the type of joints
12 robot = Manipulator(d, theta, a, alpha, revolute) #
    Manipulator object
13
14
15 # Joint limit task parameters
16 velocity_limit = [-0.5, 0.5] # Joint limits for
    joint 1
17 joint_limit_task = JointLimit("Joint limit",
    velocity_limit, [0.05, 0.15], link=1)
18
19 # End-effector task
20 sigma_endeff = np.zeros((2,1))
21 tasks = [
22     joint_limit_task,
23     Position2D("End-effector position", np.array
    ([1.0, 0.5]).reshape(2,1))
24 ]
25
26 # Simulation parameters
27 err_counter = 0
28 error_vector = np.zeros((0 ,len(tasks)))
29
30 # Simulation params
31 dt = 1.0/60.0
32
33 # Drawing preparation
34 fig = plt.figure()
35 ax = fig.add_subplot(111, autoscale_on=False, xlim
    =(-2, 2), ylim=(-2,2))
36 ax.set_title('Simulation')
37 ax.set_aspect('equal')
38 ax.grid()
39 ax.set_xlabel('x[m]')
40 ax.set_ylabel('y[m]')
41
42 line, = ax.plot([], [], 'o-', lw=2)
43 path, = ax.plot([], [], 'c-', lw=1)
44 point, = ax.plot([], [], 'rx')
45 PPx, PPy = [], []
46
47 # Simulation initialization
48 def init():
49     global sigma_endeff
50     line.set_data([], [])
51     path.set_data([], [])
52     point.set_data([], [])
53     sigma_endeff = np.random.uniform(-1.2, 1.2, (2,
    1))

```

```

54     for task in tasks:
55         if task.name == "End-effector position":
56             task.setDesired(sigma_endeff)
57     return line, path, point
58
59 def simulate(t):
60     global tasks, robot, PPx, PPy, err_counter,
        error_vector
61
62     dof = robot.getDof()
63     P = np.eye(dof)
64     err_acc = np.zeros((1,0))
65     dq = np.zeros((dof,1))
66
67     for index, task in enumerate(tasks):
68         task.update(robot)
69         J = task.getJacobian()
70         if task.isTaskActive():
71             Jbar = J @ P
72             dq += DLS(Jbar, 0.1) @ (task.
                getFeedforwardVelocity() + task.
                getGain() @ task.getError() - J @ dq
                )
73             P = P - np.linalg.pinv(Jbar) @ Jbar
74
75             new_error = robot.getJointPos(0) if task.
                name == "Joint limit" else np.linalg.
                norm(task.getError())
76             err_acc = np.block([err_acc, new_error])
77
78     robot.update(dq, dt)
79     error_vector = np.block([[error_vector], [
        err_acc]])
80     err_counter += 1
81
82     PP = robot.drawing()
83     line.set_data(PP[0,:], PP[1,:])
84     PPx.append(PP[0,-1])
85     PPy.append(PP[1,-1])
86     path.set_data(PPx, PPy)
87     point.set_data(sigma_endeff[0], sigma_endeff[1])
88     return line, path, point
89
90 # Run animation for up to 1 minute
91 animation = anim.FuncAnimation(fig, simulate, np.
        arange(0, 10, dt),
92                                interval=10, blit=
                                True, init_func=
                                init, repeat=
                                True)
93 plt.show()
94
95 # Plot errors and joint limit state after simulation
96 if err_counter > 0:
97     total_time = np.arange(0, err_counter * dt, dt)
98     fig = plt.figure()
99     ax2 = fig.add_subplot(111, autoscale_on=False,
        xlim=(0, err_counter * dt),
100                                ylim=(error_vector.min() -
        0.1, error_vector.max
        () + 0.1))
101     ax2.set_title(f'Task-Priority Control: {len(
        tasks)} Tasks, K={tasks[0].getGain()
        [0][0]}')
102     ax2.set_xlabel('Time (s)')
103     ax2.set_ylabel('Value')
104     ax2.grid()
105
106     for i, task in enumerate(tasks):
107         if task.name == "Joint limit":
108             ax2.plot(total_time, error_vector[:, i],
                label="Joint 1 Position")
109             ax2.axhline(y=velocity_limit[0], color='
                r', linestyle='dashed')
110             ax2.axhline(y=velocity_limit[1], color='
                g', linestyle='dashed')
111         else:

```

```

112         ax2.plot(total_time, error_vector[:, i],
113                 label=f"{task.name} Error")
114     ax2.legend()
115     plt.show()

```

4 RESULTS AND DISCUSSION

The result of lab is obtained as:

4.1 Exercise 1 (obstacle avoidance action)

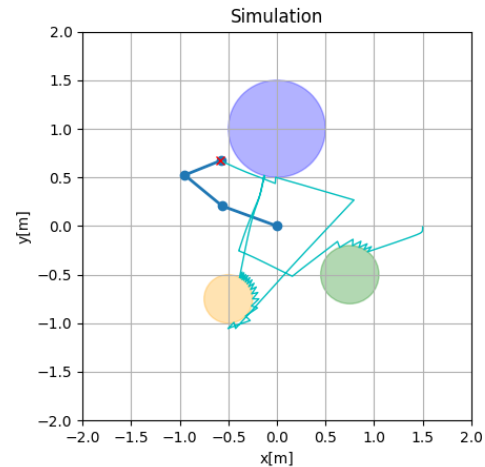


Fig. 2: Simulation of the manipulator, including end-effector goal and obstacles.

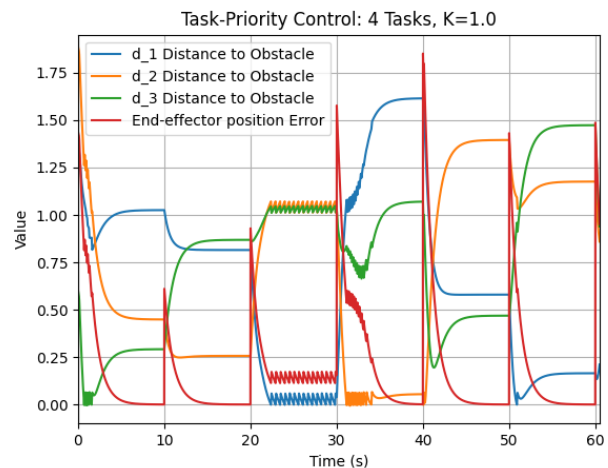


Fig. 3: Evolution of the TP control error and distance to obstacles, over time.

Fig.2: shows the simulation of the robot's motion, including the obstacles (circles), the target position, and the end-effector path. The robot dynamically adapts its trajectory to avoid collisions while trying to reach the target. Each time the end-effector comes close to an obstacle, its path bends sharply, indicating that the obstacle avoidance task has been activated and is taking control.

Fig.3: presents the time evolution of the control error and distances to each obstacle. The distance plots for obstacles d_1 , d_2 , and d_3 show multiple activations and deactivations of the avoidance tasks. As the distances fall below the activation threshold r_α , the task becomes active, and when the robot moves away and exceeds the deactivation threshold r_δ , the task deactivates, allowing the lower-priority task (end-effector positioning) to resume control.

The end-effector position error fluctuates significantly whenever an obstacle avoidance task is active, when the robot is in a safe region, the error reduces smoothly as the end-effector converges to the target.

4.2 Exercise2 (Joint limits task)

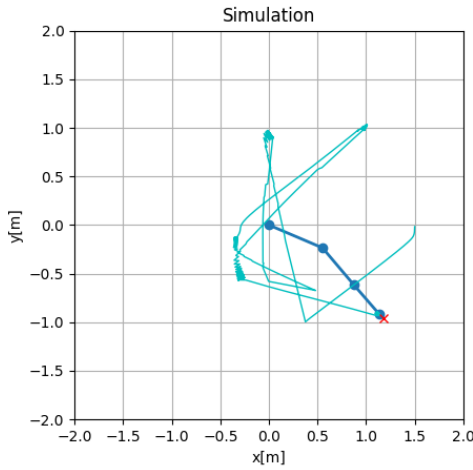


Fig. 4: Simulation of the manipulator, including end-effector goal and obstacles.

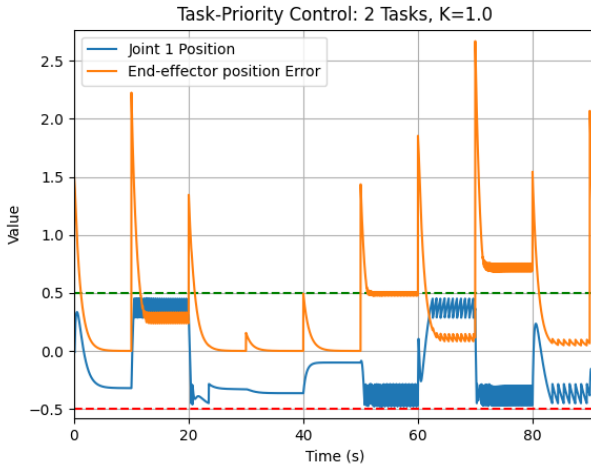


Fig. 5: Evolution of the TP control error and distance to obstacles, over time.

A joint limit constraint was introduced as a set-based task to ensure that joint 1 remains within its safe range while allowing the end-effector to reach its target.

The simulation shows that the robot successfully adapts its motion to prevent joint 1 from exceeding its limits. As

seen in the plot, the joint position remains within the defined boundaries (indicated by dashed lines), even when the end-effector task is active.

Whenever the joint approaches its limits, the joint limit task is activated, causing a temporary increase in end-effector error. Once the joint is safely back within range, the end-effector task resumes, and the error reduces.

5 CONCLUSION

In this lab, we successfully extended the Task-Priority control framework to handle inequality tasks such as obstacle avoidance and joint limits. Through simulation of a 3-link planar manipulator, we observed how these set-based tasks help ensure safety while allowing the robot to pursue its main goals. The robot was able to avoid obstacles and respect joint constraints without compromising overall task execution. These results highlight the importance and effectiveness of combining priority-based control with real-world safety constraints in robotic systems.